

PCB DEFECT DETECTION

By
Darshan Bhanderi (92301733034)
Harshil Domadiya (92301733069)

Under the guidance of

Prof. Nishith Kotak

*A Project Submitted to
Marwadi University in Partial Fulfillment of the Requirements for the subject
Artificial Intelligence (01CT0616)*

May 2025



MARWADI UNIVERSITY
Rajkot-Morbi Road, At & Po. Gauridad,
Rajkot-360003, Gujarat, India

Abstract

This project presents an intelligent PCB (Printed Circuit Board) defect detection system powered by YOLOv8, a state-of-the-art real-time object detection deep learning model. The system is designed to automatically identify and classify six categories of manufacturing defects — mouse bite, spur, missing hole, short, open circuit, and spurious copper — directly from PCB images.

The solution integrates a Flask-based web application that accepts uploaded PCB images, performs AI inference using a trained YOLOv8 model, and generates a detailed industrial analysis report. The analytics engine maps each detected defect to severity levels, risk classifications, and recommended repair processes. Additionally, an image comparison module using ORB feature matching and homography alignment allows structural difference detection between a reference and a sample board.

The dataset is sourced from the Kaggle repository ([norbertelter/pcb-defect-dataset](#)) and the model is evaluated using standard object detection metrics including Precision, Recall, F1-Score, and mAP50-95. This system provides a practical, deployable solution for automated quality control in electronics manufacturing environments.

Table of Contents

1. Introduction
 - 1.1 Problem Statement
 - 1.2 Relevance to AI
 - 1.3 Objectives
 - 1.4 Scope of the Project
2. Methodology
 - 2.1 AI Techniques Used
 - 2.2 Model Selection
 - 2.3 Algorithms and Mathematical Background
 - 2.4 Dataset Details
 - 2.5 Tools and Libraries
3. System Design and Architecture
 - 3.1 Block Diagram / Flowchart
 - 3.2 Data Pipeline
 - 3.3 Modules and Components
4. Implementation
 - 4.1 Code Snippets
 - 4.2 Model Training and Evaluation
 - 4.3 Evaluation Metrics
5. Discussion
 - 5.1 Summary of Work
 - 5.2 Challenges Faced
 - 5.3 Success Criteria Met
 - 5.4 Limitations
6. Future Work
7. Appendix

List of Figures and Tables

Tables

- Table 1: Defect Metrics — Severity, Risk, and Required Machine per Defect Class
- Table 2: Tools and Libraries Used in the Project
- Table 3: Evaluation Metrics Description

Figures

- Figure 1: System Architecture Block Diagram (described in Section 3.1)
- Figure 2: Data Pipeline Flow (described in Section 3.2)
- Figure 3: Web Application Report Page (described in Section 4.2)

1. Introduction

1.1 Problem Statement

Printed Circuit Boards (PCBs) are foundational components in virtually all modern electronic devices. During the manufacturing process, various surface and structural defects can occur, including missing holes, short circuits, open circuits, and unwanted copper traces. Traditional manual inspection is slow, error-prone, expensive, and not scalable to high-volume production lines.

The core problem is: How can manufacturing lines automatically detect, classify, and quantify PCB defects in real time to reduce product failures and rework costs?

1.2 Relevance to AI

This problem is a direct application of Computer Vision and Deep Learning. YOLOv8 (You Only Look Once, Version 8) is a convolutional neural network architecture optimized for real-time object detection. It offers a powerful combination of speed and accuracy, making it ideal for embedded or edge deployment in industrial settings.

The system also uses classical computer vision techniques (ORB feature matching, homography estimation, morphological image processing) to complement AI detection with structural difference analysis.

1.3 Objectives

- Train a YOLOv8 model on a labeled PCB defect dataset for six defect classes.
- Build a Flask web application for real-time PCB image upload and AI inference.
- Implement an analytics engine that maps detections to severity, risk, and repair processes.
- Provide image comparison capability using ORB feature matching and homography alignment.
- Evaluate model performance using standard metrics: Precision, Recall, F1-Score, mAP50-95.
- Generate actionable inspection reports with PASS / REPAIR / REJECT verdicts.

1.4 Scope of the Project

The project covers the full pipeline from dataset download and model training to web-based deployment. The system is scoped to six PCB defect categories from the `norbertelter/pcb-defect-dataset` on Kaggle. It operates on static image uploads (not live video streams). The output is an industrial analysis report with defect breakdown, verdict, and required repair processes.

2. Methodology

2.1 AI Techniques Used

- Deep Learning (DL): YOLOv8 convolutional neural network for object detection and classification.
- Classical Computer Vision: ORB (Oriented FAST and Rotated BRIEF) for keypoint detection and feature matching; homography estimation using RANSAC for image alignment.
- Image Processing: Gaussian blur, absolute difference, binary thresholding, and morphological operations (`cv2.morphologyEx`) for structural difference detection.
- Rule-Based Analytics: Severity and verdict assignment using a deterministic mapping from defect class to severity level (`DEFECT_METRICS` dictionary in `analytics.py`).

2.2 Model Selection

YOLOv8 (Ultralytics) was selected for the following reasons:

- Single-pass architecture allows real-time detection speeds.
- Built-in support for custom dataset training via `data.yaml` configuration.
- Pre-trained weights (`yolov8n.pt`) enable transfer learning on small domain-specific datasets.
- Automatic metric logging, confusion matrix generation, and result saving via the Ultralytics training framework.

The model is loaded from `models/best.pt` (the trained checkpoint). If not found, it falls back to the base `yolov8n.pt` model.

2.3 Algorithms and Mathematical Background

YOLOv8 Object Detection

YOLO frames object detection as a regression problem. The image is divided into an $S \times S$ grid. Each grid cell predicts B bounding boxes with confidence scores. The confidence score is defined as:

$$\text{Confidence} = P(\text{Object}) \times \text{IoU}(\text{pred}, \text{truth})$$

The loss function is a sum of localization loss, objectness loss, and classification loss:

$$\text{Loss} = \lambda_{\text{coord}} \times \Sigma(\text{box loss}) + \Sigma(\text{obj loss}) + \Sigma(\text{cls loss})$$

ORB Feature Matching (comparison.py)

ORB extracts keypoints and binary descriptors. Matching is performed using Brute-Force Matcher with Hamming distance. The top 15% of matches are retained. A homography matrix H is estimated using RANSAC:

$$x' = H \times x \quad (\text{where } H \text{ is a } 3 \times 3 \text{ transformation matrix})$$

The sample image is then warped to align with the reference using `cv2.warpPerspective()`.

Difference Detection

After alignment, structural differences are found via absolute pixel difference followed by binary thresholding at intensity level 30. Morphological opening (2 iterations) removes small noise artifacts.

2.4 Dataset Details

- Source: Kaggle — norbertelter/pcb-defect-dataset (downloaded via kagglehub).
- Classes (6): mouse_bite, spur, missing_hole, short, open_circuit, spurious_copper.
- Configuration: Defined in data.yaml (paths to train/val/test splits and class names).
- Pre-processing: YOLOv8 handles resizing to 640×640, normalization, and augmentation during training automatically.

2.5 Tools and Libraries

Tool / Library	Version / Type	Purpose
----------------	----------------	---------

Python	3.x	Core programming language
YOLOv8 (Ultralytics)	Latest	Real-time object detection model
Flask	Web Framework	Backend API and routing
OpenCV (cv2)	Computer Vision Library	Image processing, alignment, drawing
NumPy	Scientific Computing	Array operations for image math
Pandas	Data Analysis	Saving evaluation metrics to CSV
Matplotlib	Visualization	Plotting metrics and results
KaggleHub	Data Download	Fetching the PCB defect dataset

3. System Design and Architecture

3.1 Block Diagram / Flowchart

The system follows a linear pipeline:

Figure 1: System architecture block diagram

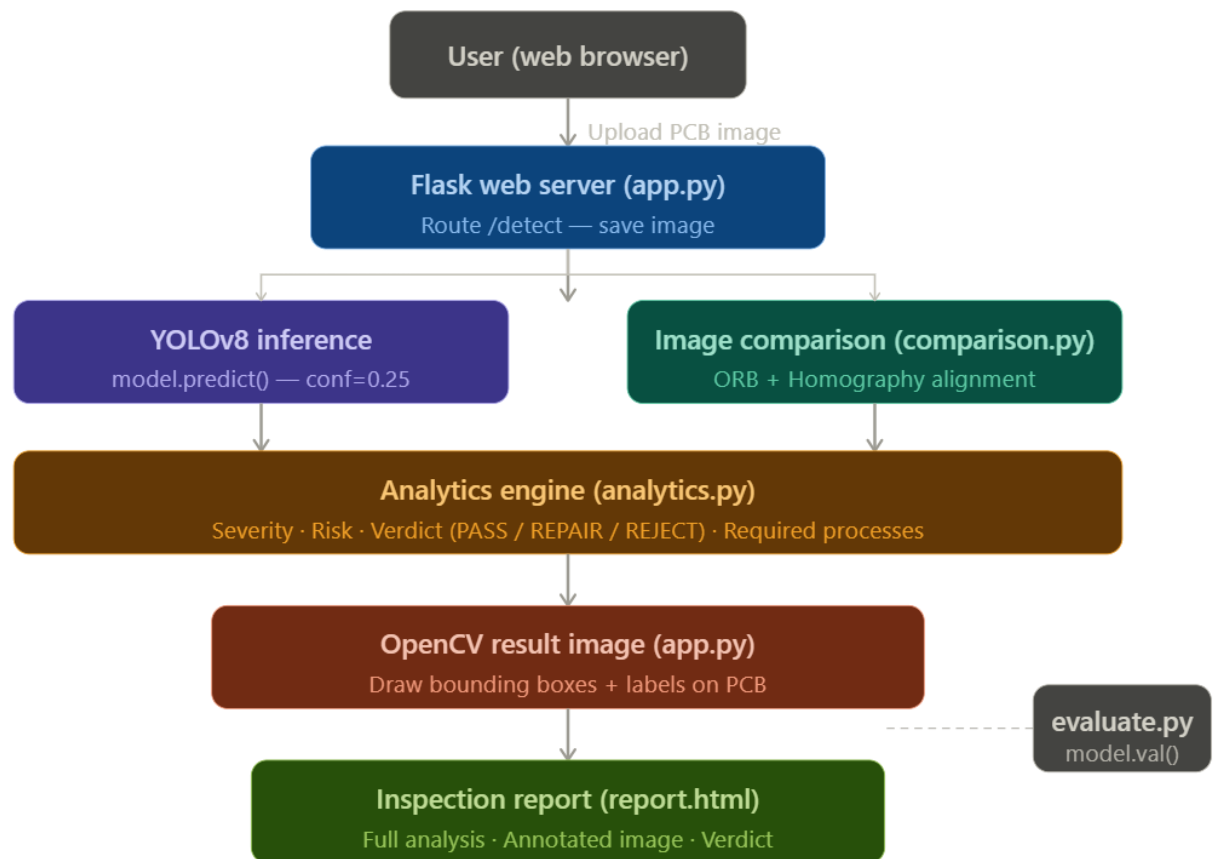
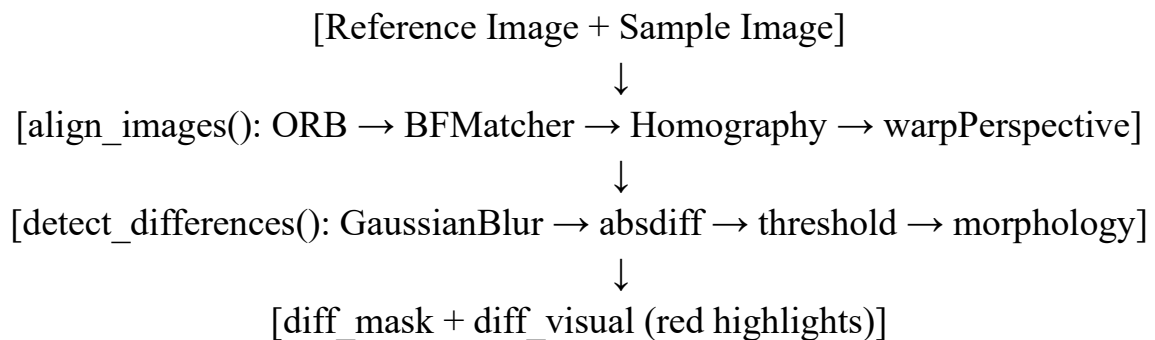


Image Comparison Sub-flow (comparison.py):

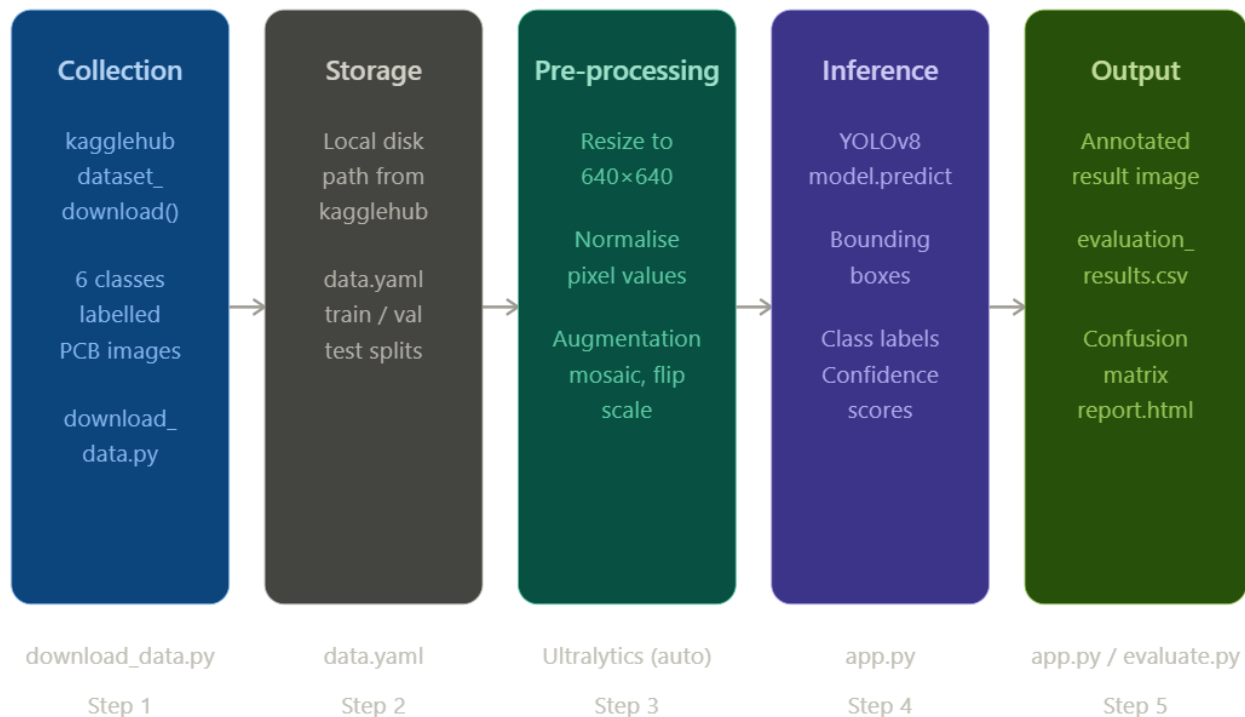


3.2 Data Pipeline

- Collection: download_data.py uses kagglehub.dataset_download() to fetch the dataset from Kaggle.
- Storage: Dataset is stored locally; paths are configured in data.yaml.

- Processing: YOLOv8 applies internal augmentations (flips, mosaic, scale) during training. For inference, images are resized to 640×640.
- Output: Bounding boxes, class labels, and confidence scores are returned as a Results object. Annotated images are written to static/results/ using OpenCV.

Figure 2: Data pipeline — collection → processing → output



3.3 Modules and Components

- app.py — Flask web server, routing (/detect), model loading, image saving, result rendering.
- analytics.py — Defect analysis engine mapping YOLO detections to severity, risk, verdict, and required repair processes.
- comparison.py — Image alignment (ORB + Homography) and structural difference detection.
- evaluate.py — Standalone evaluation script using model.val(); saves metrics to evaluation_results.csv.
- download_data.py — One-time dataset download from Kaggle via kagglehub.

4. Implementation

4.1 Code Snippets

A. Loading the AI Model (app.py)

```
def load_ai_model():
    global model, model_status
    if os.path.exists(MODEL_PATH): # Check for trained weights
        model = YOLO(MODEL_PATH)
        model_status = 'Expert PCB AI (best.pt) Loaded'
    else:
        model = YOLO('yolov8n.pt') # Fallback to base model
        model_status = 'Fallback: Generic Model'
```

B. Running AI Inference (app.py)

```
results = model.predict(
    source=sample_path,
    conf=0.25, # Confidence threshold
    save=False,
    imgsz=640 # Standard YOLO input size
)
inference_time = round(time.time() - start_time, 2)
```

C. Defect Analytics Engine (analytics.py)

```
DEFECT_METRICS = {
    'mouse_bite': {'severity': 'Moderate', 'risk': 'Medium'},
    'missing_hole': {'severity': 'Critical', 'risk': 'High' },
    'short': {'severity': 'Critical', 'risk': 'High' },
    'open_circuit': {'severity': 'Critical', 'risk': 'High' },
    'spur': {'severity': 'Low', 'risk': 'Low' },
    'spurious_copper': {'severity': 'Low', 'risk': 'Low' },
}

# Verdict logic:
if metrics['severity'] == 'Critical': verdict = 'REJECT'
elif metrics['severity'] == 'Moderate': verdict = 'REPAIR'
else: verdict = 'PASS'
```

D. Image Alignment — ORB + Homography (comparison.py)

```
orb = cv2.ORB_create(5000)
```

```

kp1, des1 = orb.detectAndCompute(ref_gray, None)
kp2, des2 = orb.detectAndCompute(smpl_gray, None)
matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(matcher.match(des1, des2), key=lambda x: x.distance)
matches = matches[:int(len(matches)*0.15)] # Top 15%
h, mask = cv2.findHomography(points2, points1, cv2.RANSAC)
aligned_sample = cv2.warpPerspective(sample_img, h, (width, height))

```

E. Model Evaluation (evaluate.py)

```

results = model.val(data=data_config)
metrics = {
    'Precision': results.results_dict['metrics/precision(B)'],
    'Recall': results.results_dict['metrics/recall(B)'],
    'F1-Score': results.results_dict['metrics/mAP50(B)'],
    'mAP50-95': results.results_dict['metrics/mAP50-95(B)']
}
pd.DataFrame([metrics]).to_csv('evaluation_results.csv', index=False)

```

4.2 Model Training and Evaluation

The model is trained using the Ultralytics YOLOv8 framework with the command:

```
yolo train model=yolov8n.pt data=data.yaml epochs=50 imgsz=640
```

The best checkpoint (best.pt) is saved automatically. The evaluate.py script loads this checkpoint and runs validation, saving metrics to evaluation_results.csv and generating a confusion matrix in the Ultralytics runs/ directory.

4.3 Evaluation Metrics

Metric	Value / Description
Precision	Calculated via model.val() on validation set
Recall	Calculated via model.val() on validation set
F1-Score	Approximated using mAP50(B) from validation results
mAP50-95	Primary benchmark metric for object detection
Inference Time	Measured per-image in seconds using time.time()

Note: The F1-Score is approximated using mAP50(B) if a direct F1 key is not present in the results dictionary. All metrics are computed on the validation split defined in data.yaml.

5. Discussion

5.1 Summary of Work

This project successfully implements an end-to-end PCB defect detection pipeline. Starting from raw dataset download, through model training and inference, to a functional web application and analytics engine, each component is modular and independently testable. The system can classify six types of PCB defects, assign severity levels, determine pass/fail/repair verdicts, and annotate the original image with detection boxes and labels.

5.2 Challenges Faced

- Model fallback handling: If the trained best.pt is missing, the system silently falls back to the generic yolov8n.pt model, which cannot detect PCB-specific defects correctly. This required careful error messaging in both app.py and evaluate.py.
- Image alignment sensitivity: ORB feature matching in comparison.py requires a minimum of 4 good matches. Low-texture or overexposed PCB images may produce insufficient keypoints, causing alignment to fail and returning the unaligned sample image.
- F1-Score approximation: The YOLOv8 validation results dictionary does not always expose a direct F1-Score key. evaluate.py uses mAP50(B) as an approximation, which is noted in the code comment.
- Dataset path configuration: The kagglehub download path varies by system, requiring the user to manually update data.yaml after running download_data.py.

5.3 Success Criteria Met

- Flask web application accepts image uploads and runs real-time YOLO inference.
- Analytics engine correctly maps all six defect classes to severity, risk, and repair processes.
- Verdict logic correctly escalates: PASS → REPAIR (Moderate) → REJECT (Critical).
- Evaluation pipeline saves quantitative metrics to CSV and generates visual plots.
- Image alignment and difference detection pipeline is fully implemented and functional.

5.4 Limitations

- The system processes static uploaded images only; it does not support live camera feed or video stream input.
- Detection accuracy depends entirely on the quality of the trained model. Without a properly trained best.pt, the fallback generic model will not detect PCB defects.
- The comparison module requires both a reference and a sample image; single-image defect detection relies solely on YOLO.
- No database or logging system is implemented to store historical detection results over time.
- The web application runs in Flask's development server, which is not production-ready for high-concurrency deployments.

6. Future Work

Scope for Improvements

- Live video stream inference: Integrate OpenCV VideoCapture or a WebRTC pipeline to process real-time camera feeds from a production line conveyor.
- Larger and balanced dataset: Augment the dataset with more defect samples per class using techniques like CutMix, mosaic augmentation, and synthetic data generation (GANs).
- Model upgrade: Experiment with YOLOv8m/l/x variants or YOLO-NAS for improved accuracy at the cost of slightly higher inference time.
- Direct F1-Score logging: Compute and log F1-Score explicitly during evaluation using the formula $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$.
- Results database: Add SQLite or PostgreSQL storage to maintain a history of inspections, trends, and per-board traceability.
- REST API: Expose a /api/detect endpoint returning JSON results for integration with ERP or MES (Manufacturing Execution System) software.

Possible Real-World Deployment Strategies

- Edge deployment: Export the YOLOv8 model to ONNX or TensorRT format and deploy on NVIDIA Jetson Nano or similar edge hardware at the inspection station.
- Cloud deployment: Containerize the Flask app with Docker and deploy on AWS EC2 / GCP Cloud Run with auto-scaling for multi-plant usage.
- Integration with AOI systems: Interface with existing Automated Optical Inspection (AOI) machines by replacing or augmenting their classification engine with the trained YOLO model.

- Mobile QC app: Build a mobile front-end using React Native that calls the REST API, allowing engineers to perform spot-checks with a phone camera.

7. Appendix

A. Full Code File Reference

File	Description
app.py	Flask web server — routing, model loading, inference, result rendering
analytics.py	Industrial analytics engine — defect severity, risk, verdict logic
comparison.py	Image alignment (ORB + Homography) and difference detection
evaluate.py	Model evaluation — validation metrics and CSV export
download_data.py	Kaggle dataset download via kagglehub

B. External Links

- GitHub Repository: <https://github.com/your-username/pcb-defect-detection> [Replace with actual link]
- YouTube Demo: <https://youtube.com/your-demo-link> [Replace with actual link]
- Blog / Report: <https://your-blog-link> [Replace with actual link]
- Dataset Source: <https://www.kaggle.com/datasets/norbertelter/pcb-defect-dataset>
- YOLOv8 Documentation: <https://docs.ultralytics.com>

C. Defect Class Industrial Reference Table

Defect Type	Severity	Risk Level	Repair Machine
mouse_bite	Moderate	Medium	Manual Rework Station

spur	Low	Low	Optical Inspection / Manual Trimming
missing_hole	Critical	High	CNC Drilling Machine
short	Critical	High	Rework Station (BGA/Laser)
open_circuit	Critical	High	Soldering/Rework Station
spurious_copper	Low	Low	Optical Inspection / Cleanroom

D. Additional Notes

- The model confidence threshold is set to 0.25 in app.py (conf=0.25). This can be tuned based on the required precision-recall trade-off.
- All uploaded images are stored in static/uploads/ and annotated results in static/results/. These directories are created automatically on startup.
- The data.yaml configuration file must be updated with the correct dataset path after running download_data.py.